

UD9: BASES DE DATOS DISTRIBUÍDAS

Índice

Introducción.....	2
Bases de datos distribuidas.....	2
Fragmentación vertical.....	2
Fragmentación horizontal.....	3
Fragmentación mixta.....	3
Consultas distribuidas.....	4
Bases de datos Replicadas.....	7

Introducción

En la actualidad, cuando las bases de datos empiezan a estar alojadas en sitios remotos, la tendencia es la distribución de los datos en servidores remotos y distribuidos. Disponer de bases de datos de alta disponibilidad se consigue con técnicas asociadas a la replicación de servidores. En arquitecturas mixtas, las técnicas de fragmentación son la clave para el diseño de bases de datos distribuidas. Las consultas se rediseñan para proveer mecanismos abstractos sobre la ubicación real de los datos y entran en acción la explotación de vistas y las vistas materializadas.

Bases de datos distribuidas

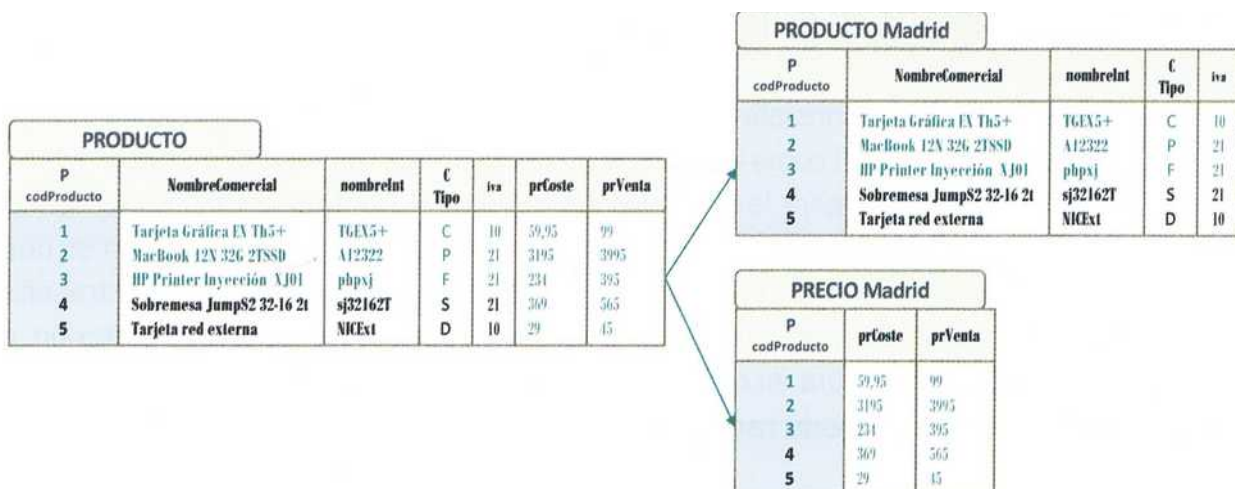
Las bases de datos distribuidas son soportadas por más de un servidor. Estos servidores requieren de una arquitectura física y lógica que permita la conexión entre ellos. Los sistemas gestores de bases de datos incorporan mecanismos para dar soporte a la distribución de datos en diferentes máquinas. Cuando un usuario accede a los datos, no se conoce la capa de resolución de datos distribuidos, sino que los datos se obtienen como si estuvieran en una única máquina. Esta abstracción posibilita nuevos beneficios, pero se deben tener en cuenta los riesgos en coste que supone implementar la arquitectura



La fragmentación de las tablas se puede hacer **horizontal** o **verticalmente**.

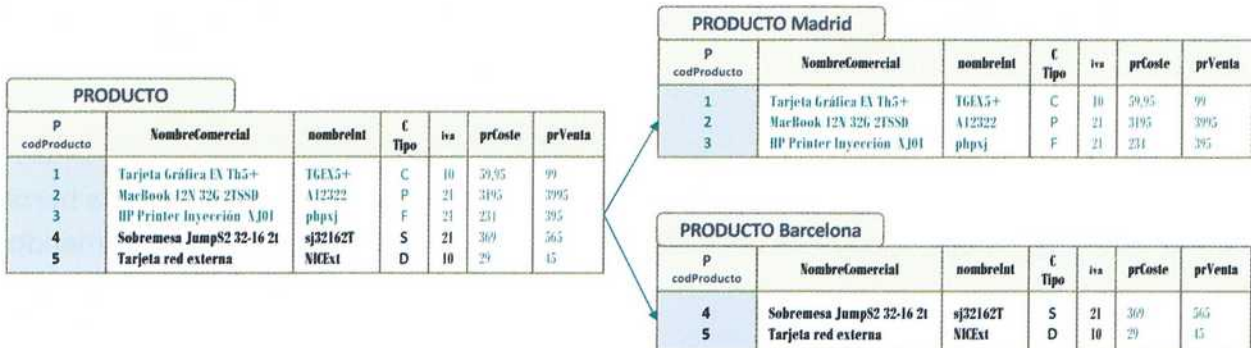
Fragmentación vertical

Cuando se emplea la fragmentación vertical o de columna, cada fragmento se obtiene a través de la selección de las columnas que componen el fragmento. El **join** de estos fragmentos constituye la tabla original.



Fragmentación horizontal

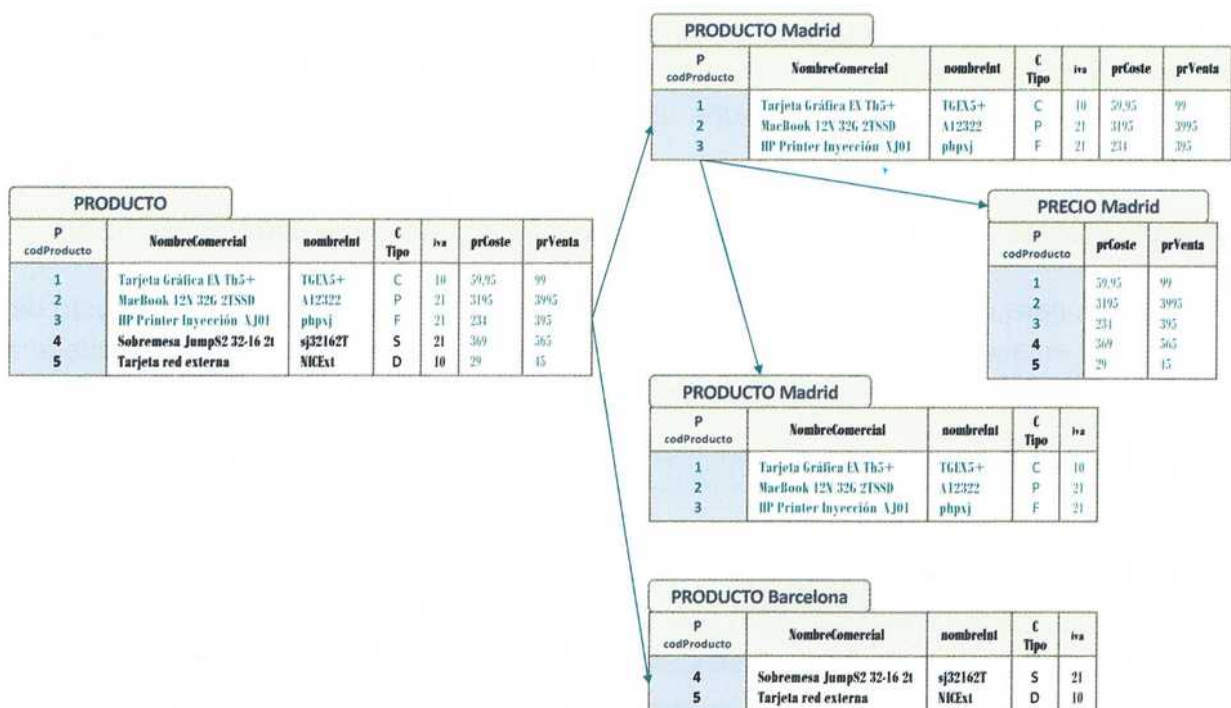
La fragmentación horizontal o de fila, afecta a las diferentes filas de una tabla. Estas filas pueden estar en diferentes máquinas. Para la composición de la tabla original se debe unir cada subconjunto de filas. Para ello se aplica la operación **unión** de cada fragmento.



En este caso todas las bases de datos (la local y la/s remota/s) tienen el mismo esquema de datos (las mismas tablas con los mismos atributos). Cada base de datos tendrá un subconjunto de los datos. Esta distribución es buena para bases de datos de sucursales donde cada una tiene la misma aplicación de gestión como un conjunto de farmacias. Cada sucursal tiene tablas con sus clientes, productos, etc, pero se puede acceder de alguna manera a las otras sucursales para saber si tienen tal o cuál producto en stock.

Fragmentación mixta

Estas dos técnicas pueden usarse simultáneamente sobre tablas que deben ser fragmentadas tanto por columnas como por filas. Esta fragmentación se denomina fragmentación **mixta**.



Consultas distribuidas

La distribución de consultas consiste en capacitar al sistema de aplicaciones con el acceso tanto a su base de datos local como a aquellas bases de datos remotas que este requiera.

Una técnica que usa Oracle para las conexiones remotas y ejecutar consultas de forma transparente es el uso de enlaces a bases de datos. Cada enlace es una conexión en una dirección a una máquina remota. Para ello, se requiere de un usuario y una contraseña conocidos y la cadena de conexión que permite la comunicación. La cadena de conexión, o nombre de servicio, se configura en el fichero tnsnamesora. El siguiente texto corresponde a un nombre de servicio llamado nsmad que se añade al final del fichero tnsnames.ora:

```
CANGAS =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST = 192.168.143.43)(PORT=1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = XEPDB1)
    )
  )
```

Para comprobar la conectividad por medio de un nombre de servicio se puede usar la herramienta tnsping. Desde la consola del sistema operativo se escribe el siguiente comando: **tnsping cangas**

Este comando comprueba la conectividad usando el adaptador necesario para resolver el alias creado en tnsnamesora.

Una vez que en un servidor se crea el nombre del servicio, se debe crear la comunicación entre el cliente y el servidor. El comando que debe usarse es

CREATE DATABASE LINK.

Un enlace será público, PUBLIC, cuando se pretenda que todos los usuarios puedan usarlo. Si se usa la opción **CONNECT TO nombreUsuario IDENTIFIED BY password**, el enlace se crea para ese usuario. Sin embargo, si no ponemos nada el enlace se puede usar con el usuario actual que esté haciendo la query.

El siguiente ejemplo crea una conexión usando el usuario userCangas a través del nombre de servicio cangas:

```
CREATE DATABASE LINK LinkCANGAS CONNECT TO userCangas
IDENTIFIED BY password USING 'CANGAS';
```

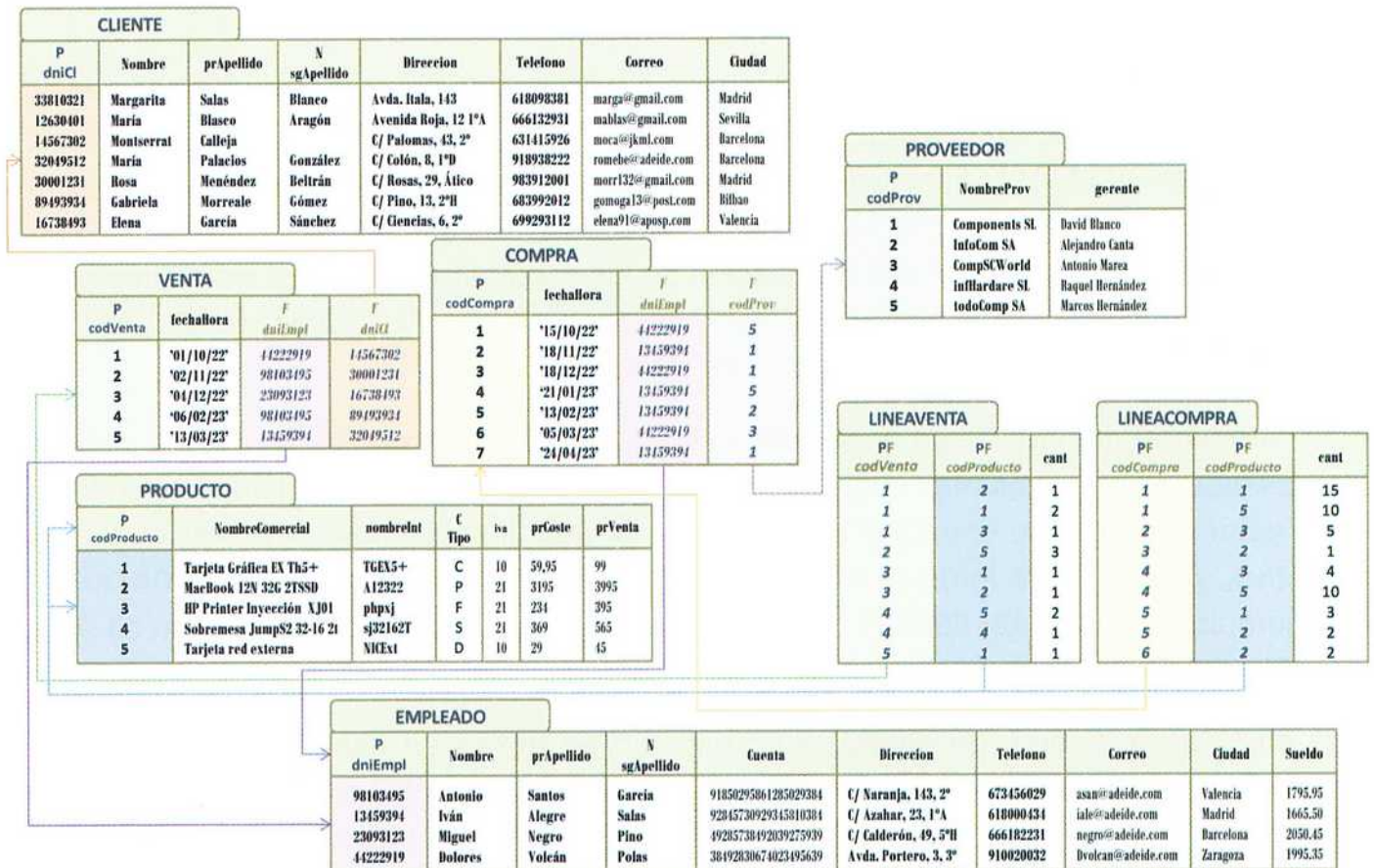
Finalmente, para usar una tabla remota en una consulta se usa el simbolo @ seguido del nombre del enlace creado con CREATE DATABASE LINK. La siguiente consulta accede a una base de datos remota y a la tabla Producto alojada en ella:

```
SELECT * FROM Producto@LinkCangas;
```

PRÁCTICA

Este es un trabajo en grupo de 3.

Cada uno de vosotros creará un nodo con una pluggable llamada tienda



Supón que la organización ha decidido distribuirla y alojar una en Madrid y otra en Vigo. Estos serán los otros dos compañeros.

Crea una cuenta de usuario en cada servidor que se usará para las conexiones. Se llamarán userVig y userMad respectivamente.

En cada servidor, añade en tnsnames.ora la descripción correspondiente a todos los otros servidores. A continuación crea los enlaces necesarios para la comunicación bidireccional de todos los servidores.

La organización ha decidido que

a) La sede Central se encarga de:

a) Gestionar todas las compras, por tanto, gestiona el alta de todos los productos.

b) Los empleados de todas las tiendas son dados de alta aquí por lo que necesitamos un campo que indique en qué sede trabaja.

c) Creará una secuencia para la clave de las ventas que está distribuida por nodos

d) Creará vistas de unión para los clientes y ventas

b) La sede de Madrid se encarga de :

a) El stock de Madrid por lo que habrá que crear un campo stock en la tabla productos.

b) Los clientes de Madrid.

c) Las ventas realizadas en Madrid son gestionadas por Madrid.

d) Creará vistas con join entre la tabla productos de la central y la suya para obtener la tabla completa.

b) La sede de Vigo se encarga de:

a) Los clientes y ventas del resto de España

b) El stock de Vigo se gestiona aquí por lo que añadimos el campo stock.

c) Creará vistas con join entre la tabla productos de la central y la suya para obtener la tabla completa.

Pinta con colores las tablas/campos que irán en cada uno de los tres nodos del sistema distribuido. Crea en cada nodo las tablas/vistas necesarias para:

En el nodo central:

Tener un catálogo total de todos los productos junto con el stock de la sede en que están.

Tener un catálogo total de todas las ventas producidas en todas las sedes

En cada nodo Madrid/Vigo

Tener una vista productos con los productos de la sede con sus precios y stock

Tener una vista de empleados con los empleados de la sede

Piensa en un select que se haría desde un nodo a la central para buscar un producto por si se ha acabado en la sede y queremos ver si está en la otra.

Bases de datos Replicadas

Las bases de datos replicadas están orientadas a la alta disponibilidad, ya que el objetivo es disponer de la misma base de datos en dos ubicaciones diferentes. A diferencia de las bases de datos distribuidas, las replicadas suponen tener dos bases de datos idénticas, por lo que la redundancia de datos es máxima al estar todos los datos almacenados dos veces. A la base de datos que se decide replicar se le llama base de datos maestra y la resultante de la replicación se llama base de datos esclava. En una arquitectura de alta disponibilidad es posible que se disponga de más de un servidor esclavo. La caída de uno de ellos supone disponer de los datos sin ningún problema.

El mantenimiento de las bases de datos replicadas es altísimo. Cada inserción, actualización y eliminación debe ser contemplada en todas las bases de datos y hacerse sin perder coherencia. Este tipo se usa cuando se dispone de una tasa de comunicación entre servidores muy alta y de máquinas realmente capaces para su explotación.

Generalmente, estas arquitecturas se aplican en sistemas en los que la mayoría de las operaciones son de consultas y el nivel de actualización de los registros no es excesivamente alto. De este modo, los servidores esclavos balancean la carga de las operaciones de consulta distribuyendo el trabajo de resolución.

Comunicación asíncrona: los cambios que se van a realizar son almacenados en el servidor maestro y cada cierto tiempo estipulado, estos son enviados a todos los servidores esclavos que procedan. En los servidores se deben acometer todas las transacciones pendientes, ya sean en los registros redo logs alojados en la caché correspondiente, en los ficheros de actualización o en los ficheros de actualización archivados. Una vez que sea necesario, las transacciones serán volcadas a los ficheros de datos correspondientes.

Comunicación sincrónica: cada transacción que se almacena en el servidor maestro es enviada a los servidores esclavos para que estos la procesen según necesiten. El objetivo final es que cada acción llevada a cabo en el servidor maestro sea replicada en los servidores esclavos.

PRÁCTICA

Implementación con Vistas materializadas

Una técnica para replicar las tablas que componen una base de datos es el empleo de vistas materializadas. Estas vistas se ejecutan cada cierto tiempo, actualizando sus registros. Así, en un servidor esclavo, se puede tener una vista materializada para cada tabla del servidor maestro.

Implementa vistas materializadas en los nodos con los productos del otro y establece su política de refresco.